WEEK 11 - PYTHON PROGRAMMING

From Design to Running Code

Make it work first. Then make it right.

ACT 1 · PROTOTYPE
run it first

ACT 2 · STRUCTURE
fill the skeleton

ACT 3 · REFACTOR
make it right

LAB
ship F1

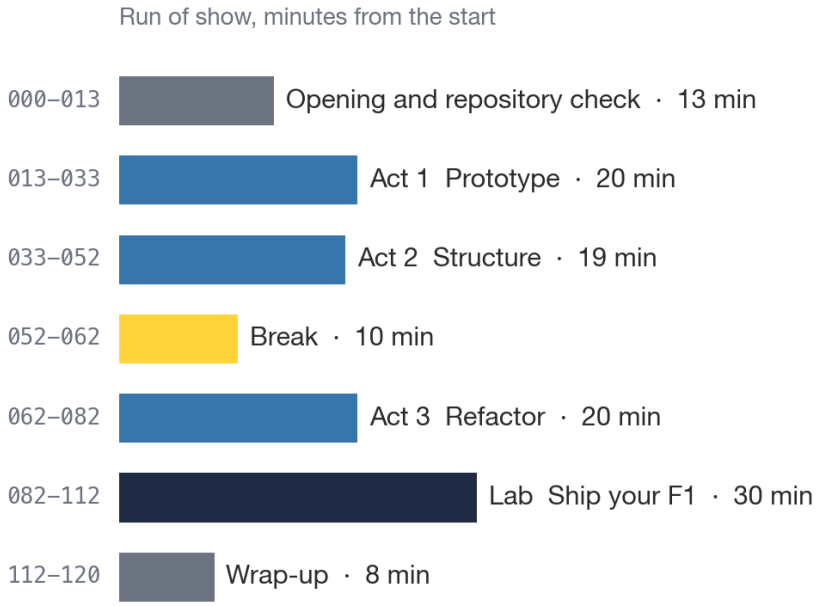
120 minutes, including a break

Instructor: MinGi KYUNG

The Plan: Run Something Today

Last week you drew the program and gave it a home. Today it starts to run.

- Prototype: the thinnest version that runs end to end, fakes included.
- Structure: fill the skeleton one contract at a time, I/O still at the edges.
- Refactor: same behavior, better shape, checks green before and after.
- Lab: your F1 on a feature branch, from stub to merged pull request.

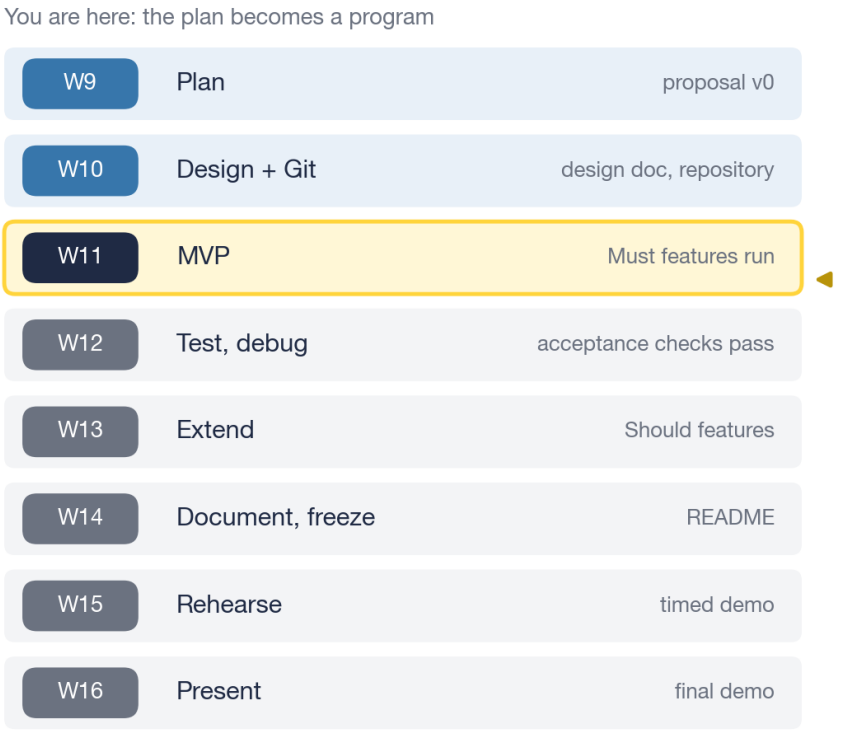


Where We Are in the Project

Week 10 left a skeleton with contracts.
Week 11 makes it real.

- Week 10: design document, repository, every function a stub with a docstring.
- Week 11: Must features run end to end. Today F1, next session F2 and F3.
- Week 12: tests and debugging against a program that already runs.

RULE MVP deadline is the end of next session: every Must feature runs from python3 main.py.



Check Your Repository First

Five minutes. Today's lab starts from this state, so close the gaps now.

- git switch main, then git pull: your laptop matches GitHub.
- git status says: nothing to commit, working tree clean.
- python3 main.py runs without a traceback. Printing TODO is fine.
- Every function in your design has a stub, with the contract as its docstring.
- Your design document is open beside VS Code.

> **TIP** Missing a stub? Add it now with raise NotImplementedError("name"). That is a legal body.

Two Ways to Spend Week 11

Same design, same hours. One student has a demo by Friday.

- Layers: finish all of storage.py, then all of logic.py, then report.py, then main.py.
- Nothing runs until the last file is done. Every bug appears at once, on Thursday night.
- Slices: make F1 run through all four files first, ugly but complete. Then F2. Then F3.
- Something runs after every session.



PREDICT Which student finds the csv header bug first, and how long do they search?

A finds the csv header bug on Thursday, in 200 lines.

B finds it on Monday, in the 12 lines of append_record.

ACT 1 · PROTOTYPE

Prototype: Make It Run First

A program that runs badly teaches more than a design that sits perfectly.

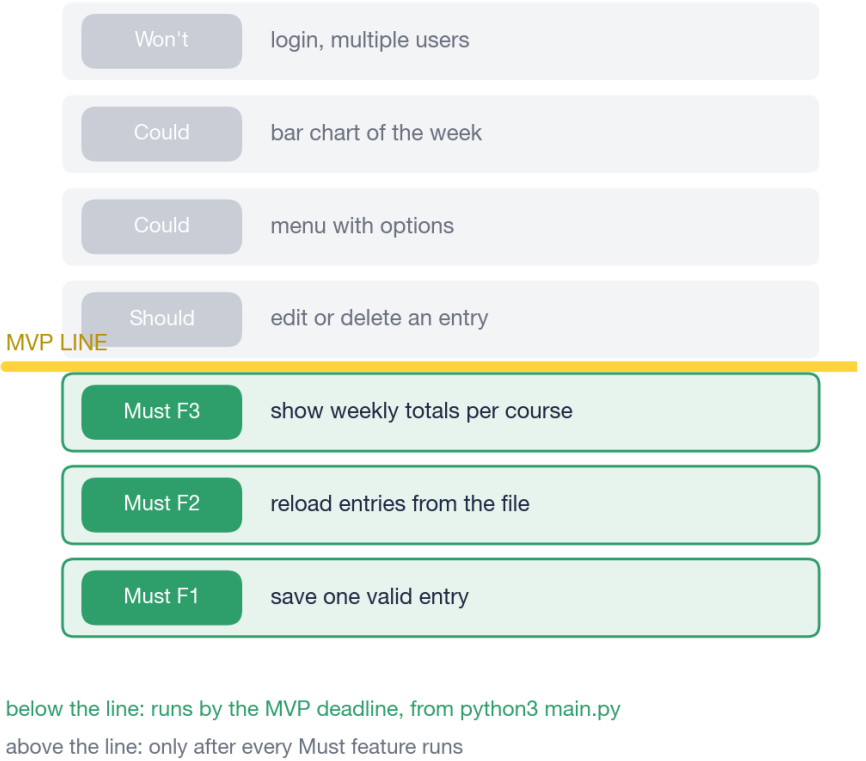
- What MVP means for your project, and what it leaves out.
- Fake what you do not have yet, so the whole pipeline runs today.

What MVP Means Here

Minimum, viable, product. Three words, three tests for your plan.

- Minimum: the Must features and nothing else. No menu, no colors, no charts.
- Viable: a real user does the job once, start to finish, without you explaining.
- Product: python3 main.py on any laptop.
- Study logger MVP: type one entry, it is saved, the weekly totals print. That is all.

TRAP "I'll add F1 after the menu works." The menu is a Could. Build the slice first.

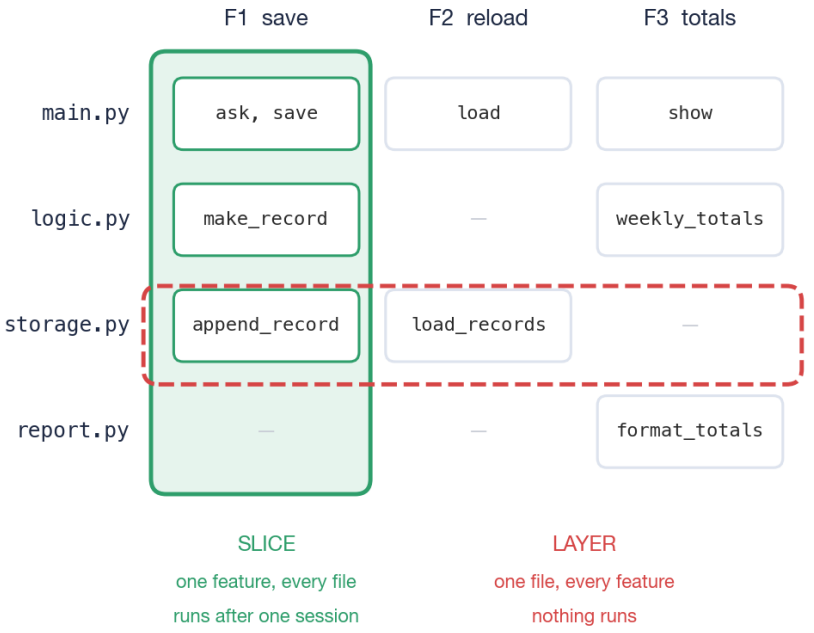


Slices, Not Layers

A slice is one feature through every file. A layer is one file through every feature.

- F1 touches main.py, logic.py and storage.py. Build those three pieces, then run.
- Every slice ends with a run: python3 main.py does the job for real.
- A layer ends with nothing to run.
- The grader's log sees one branch per slice. That is the Git story week 10 asked for.

PAYOFF The order of your slices is the order of your Must features. The plan is already written.



Fake It, Then Make It

ACT 1 · PROTOTYPE

The pipeline runs on day one. Every fake is a TODO you can see on the screen.

- A fake is a function that returns a constant and ignores its arguments.
- The real contract stays in the docstring. Only the body is pretending.
- Run it. The numbers are wrong, but the shape of the program is right.

PYTHON

```
FAKE_ROWS = [{"course": "python", "minutes": 90}]

def load_records(path):
    return FAKE_ROWS          # fake: no file yet

def weekly_totals(records):
    return {"python": 90}     # fake: not summing

def format_totals(totals):
    return [f"{c} {m} min"
            for c, m in totals.items()]

rows = load_records("data/study.csv")
for line in format_totals(weekly_totals(rows)):
    print(line)
```

OUTPUT

```
python 90 min
wrong numbers, right pipeline
```

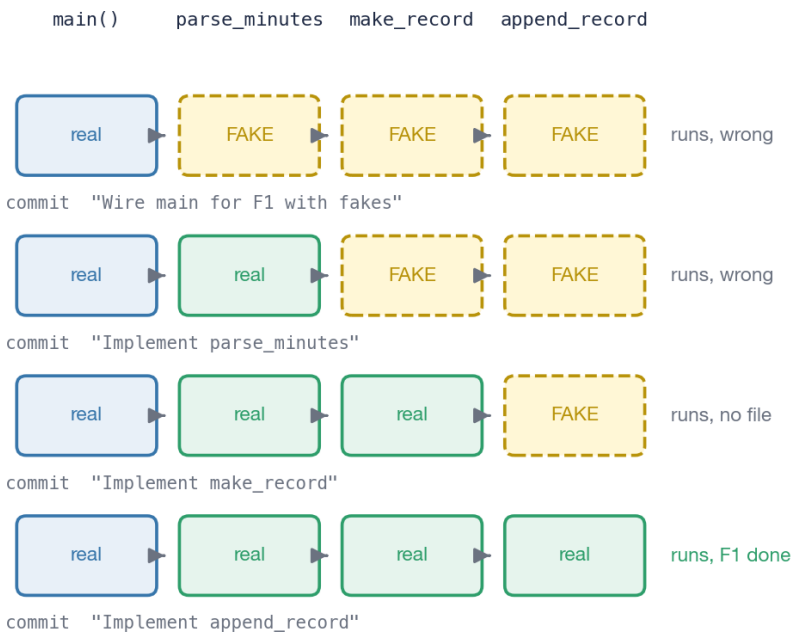
Replace One Fake at a Time

ACT 1 · PROTOTYPE

Inside a slice, work from the core outward.
Each step leaves the program running.

- Logic first: no file, no keyboard, so one check proves it in a second.
- Storage next: one function opens the file, with the real header and newline rules.
- Then wire main to the real names, delete the fake, run the whole thing.
- Then: run the checks, run main, commit.

RULE Never two fakes in one step. When it breaks, you want exactly one suspect.



one fake replaced per step: when it breaks, there is one suspect

Spike in Scratch, Build in the Repo

ACT 1 · PROTOTYPE

A spike is a throwaway experiment that answers one question. Then it is deleted.

- Question: how does csv.DictWriter end a row? Ten lines in scratch/ answer it.
- The answer goes into your real function. The spike goes in the bin.
- Spikes break every rule: print everywhere, no docstring. Fine. They are never committed.
- scratch/ is in .gitignore, so a spike cannot slip into a pull request.

PYTHON

```
# scratch/spike_csv.py: one question, then delete
import csv, io

FIELDS = ["date", "course", "minutes"]
buffer = io.StringIO()
w = csv.DictWriter(buffer, fieldnames=FIELDS)
w.writeheader()
w.writerow({"date": "2026-11-10",
            "course": "python", "minutes": 30})
print(repr(buffer.getvalue()))
```

OUTPUT

```
'date,course,minutes\r\n2026-11-10,python,30\r\n'
rows end in \r\n: open files with newline=""
```

Make It Work, Make It Right, Make It Fast

ACT 1 · PROTOTYPE

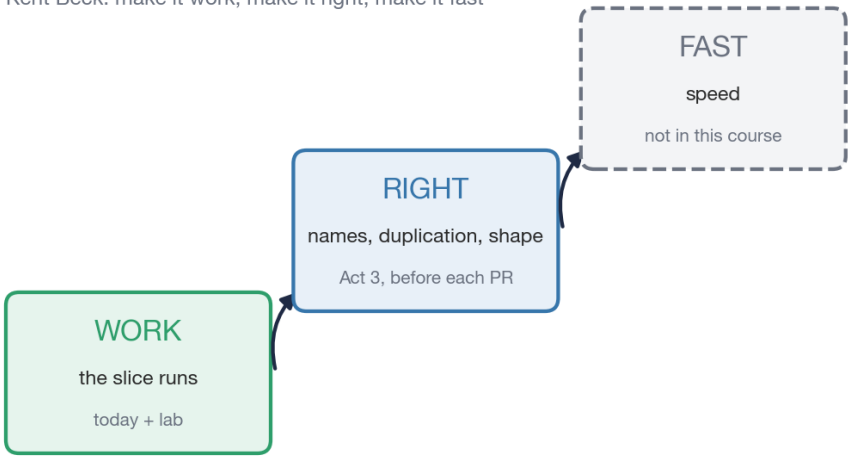
Three jobs, in this order. Doing them at once does none of them.

- Work: the slice runs end to end. Today, and in the lab.
- Right: names, duplication, structure. Act 3, and before every pull request.
- Fast: speed. Not in this course. A 20-row CSV needs no optimizing.

TRAP "Let me make this loop efficient first." Nothing runs yet. Efficiency of nothing is nothing.

In this order. One at a time.

Kent Beck: make it work, make it right, make it fast



Practice 1: Draw Your F1 Slice

ACT 1 · PROTOTYPE

Four minutes, on paper. Your project, from your design document.

- List the functions F1 calls, in call order, with their modules.
- Mark the one you will fake first, and the constant it returns.
- Write the proof that F1 works: the acceptance criterion from week 9.
- Your lab plan: one commit per line.

OUTPUT

```
F1  <your feature sentence>
 1. main()           asks      main.py
 2. make_record()    checks    logic.py   fake first
 3. append_record()  writes    storage.py
 proof: run main, then one new row in the file
```

> **TIP** What to fake first? The function whose real body needs a file or the keyboard.

ACT 2 · STRUCTURE

Structure: Fill the Skeleton

Ugly inside a function is allowed. Ugly between functions is not.

- From stub to body, one contract at a time, with a check after each.
- Keep the week 10 shape while you type: logic raises, main talks, storage opens.

From Stub to Body

The docstring was written in week 10.
Today you write the lines under it.

- Read the contract: what goes in, what comes out, what fails and how.
- Write the simplest body that keeps the promise. Simple beats clever.
- Prove it with one line: an assert for the good case.

OUTPUT

```
parse_minutes keeps its contract
```

BEFORE

```
def parse_minutes(text):
    """Return text as a positive whole number
    of minutes. Raises ValueError if text is
    not a whole number or is not above 0."""
    raise NotImplementedError("parse_minutes")
```

AFTER

```
def parse_minutes(text):
    """Return text as a positive whole number
    of minutes. Raises ValueError if text is
    not a whole number or is not above 0."""
    minutes = int(text)
    if minutes <= 0:
        raise ValueError("minutes must be positive")
    return minutes

assert parse_minutes("90") == 90
print("parse_minutes keeps its contract")
```

Storage: One Function Opens the File

append_record is the only code that knows the file format. Everyone else sees dicts.

- newline="" because the spike showed rows end in \r\n. csv handles line endings itself.
- Mode "a" appends. The header is written once, only when the file is new.
- FIELDS is the one place the column order lives. storage.py owns it.

OUTPUT

```
date,course,minutes
2026-10-24,python,90
2026-10-25,stats,45
2026-11-10,python,30
```

PYTHON

```
import csv
from pathlib import Path

FIELDS = ["date", "course", "minutes"]

def append_record(path, record):
    """Append one record; write the header
    first if the file is new."""
    is_new = not Path(path).exists()
    with open(path, "a", newline="") as f:
        w = csv.DictWriter(f, fieldnames=FIELDS)
        if is_new:
            w.writeheader()
        w.writerow(record)

record = {"date": "2026-11-10",
          "course": "python", "minutes": 30}
append_record("data/study.csv", record)
print(open("data/study.csv").read())
```

Wire main() for F1 Only

main asks, calls, reports. For F1 that is three questions and one try.

- The except branch turns logic's ValueError into a sentence for the user.
- No F2, no F3 yet. The slice is F1.
- Run it from the project folder. Then open data/study.csv and look.

OUTPUT

```
$ python3 main.py
Date (YYYY-MM-DD): 2026-11-10
Course: python
Minutes: 30
Saved.
```

PYTHON

```
from logic import make_record
from storage import append_record

DATA = "data/study.csv"

def main():
    date = input("Date (YYYY-MM-DD): ")
    course = input("Course: ")
    minutes = input("Minutes: ")
    try:
        record = make_record(date, course, minutes)
    except ValueError as error:
        print("Not saved:", error)
    else:
        append_record(DATA, record)
        print("Saved.")

if __name__ == "__main__":
    main()
```

TRAP: The Shortcut That Leaks

"It is just a prototype, so I printed the error and returned None." Two minutes later:

- The caller trusted the contract: a dict comes back. It got None.
- Here the crash is one line away. In your project it is three files away.
- Prototype inside the body all you like. The boundary keeps its promise: a dict, or a raise.

PYTHON

```
def make_record(date, course, minutes_text):
    """Return one record dict. Raises ValueError
    if minutes is not positive."""
    minutes = int(minutes_text)
    if minutes <= 0:
        print("Not saved: minutes must be positive")
        return                  # "just for now"
    return {"date": date, "course": course,
            "minutes": minutes}

record = make_record("2026-11-10", "python", "-30")
print("Saving", record["course"])
```

OUTPUT

```
Not saved: minutes must be positive
TypeError: 'NoneType' object is not subscriptable
the boundary lied, and the caller paid
```

Run, Then Commit

One step, one run, one commit. The log reads like your design document.

- test_logic.py proves the core. main.py proves the slice. Both, before every commit.
- The message names the function, so the grader sees the slice being built.
- Small commits are free undo points. One big commit is a bet.

SHELL

```
python3 test_logic.py
python3 main.py
git add -A
git commit -m "Implement parse_minutes"
```

OUTPUT

```
$ git log --oneline
9c7971e Implement append_record
ebc447d Implement make_record
8a220b6 Implement parse_minutes
6836c17 Wire main for F1 with fakes
da9b65f Add project skeleton
```

Grow test_logic.py with Every Function

One check per logic function, before you move on. Week 12 will thank you.

- The good case as an assert. The bad cases in a loop that must raise.
- Only logic is tested here: no file, no keyboard, finished in a second.
- Run it before every commit. It is the first line of the lab checklist.

PAYOFF Week 9 criteria, week 10 contracts, week 11 checks: the same sentences.

PYTHON

```
from logic import make_record, parse_minutes

assert parse_minutes("90") == 90
rec = make_record("2026-11-10", " Python ", "30")
assert rec == {"date": "2026-11-10",
               "course": "python", "minutes": 30}
for bad in [("2026-11-10", "", "30"),
            ("2026-11-10", "python", "-30")]:
    try:
        make_record(*bad)
    except ValueError:
        pass
    else:
        raise AssertionError(f"{bad} was accepted")
print("All checks passed.")
```

OUTPUT

```
All checks passed.
```

Done Means Running

A slice is done when every line below is true. Not before, and not "almost".

RULE "Almost done" is a fake. Either the row is in the file or it is not.

OUTPUT

```
F1 is done when
[x] python3 test_logic.py -> All checks passed.
[x] python3 main.py saves a valid entry
[x] a bad entry prints Not saved, changes nothing
[x] no input(), print() or open() in logic.py
[x] every step is a commit on feature/f1-...
[x] the branch is pushed, the pull request is open
```

BREAK

Halftime

Ten minutes. When we return: the same program, a better shape.

- After the break: refactoring, a catalog of five small moves, and a tidy-up practice.
- Then the lab: your F1, from stub to merged pull request.

ACT 3 · REFACTOR

Refactor: Same Behavior, Better Shape

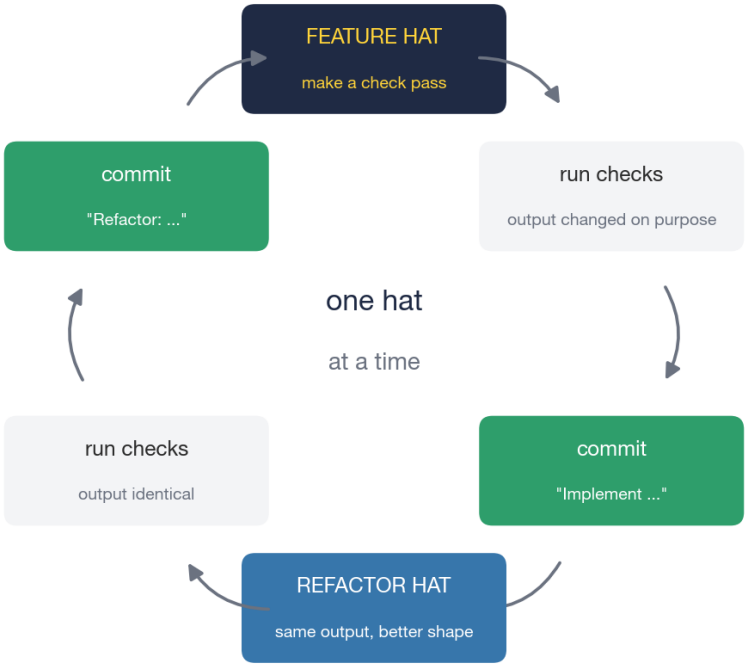
Change how the code is written without changing what it does.

- A catalog of five small moves, each one commit.
- Checks green before, checks green after. Otherwise it is not refactoring.

Two Hats

Adding behavior and improving shape are different jobs. Wear one hat at a time.

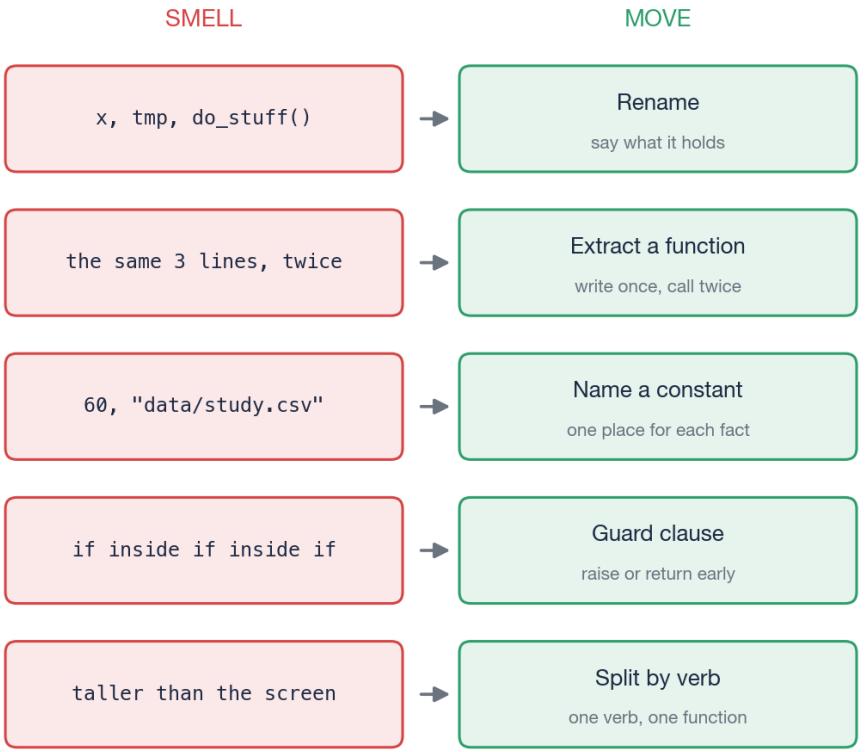
- Feature hat: make a check pass that failed. Output changes on purpose.
- Refactor hat: output identical, structure better. The checks prove it stayed identical.
- Switch hats only at a commit. Never "while I'm in here".
- In this course: feature hat in the lab, refactor hat before every pull request.



Five Smells, Five Moves

You do not need a textbook. These five cover nearly every student project.

- A name that says nothing (x, data, tmp, do_stuff): rename.
- The same three lines twice: extract a function, call it twice.
- A bare number or string that means something (60, "data/study.csv"): name a constant.
- An if inside an if inside an if: guard clauses, return or raise early.
- A function taller than your screen: split by the verb test from week 10.



one move, one run of the checks, one commit

Before: It Works. Now Read It Aloud

ACT 3 · REFACTOR

Every check passes. Say what this function does in one sentence.

- What is t? What is k? What does show show?
- Two branches, two f-strings. Read them side by side. How do they differ?
- You cannot answer quickly. That is the smell. Not a bug, a cost.

PREDICT Are the two branches really different? Compute 45 // 60 and 45 % 60.

BEFORE

```
def show(t):
    out = []
    for k in sorted(t, key=t.get, reverse=True):
        v = t[k]
        if v >= 60:
            out.append(f"{k:<8}{v:>4} min  "
                       f"{v // 60}h {v % 60:02d}m")
        else:
            out.append(f"{k:<8}{v:>4} min  "
                       f"0h {v:02d}m")
    return out

for line in show({"python": 135, "stats": 45}):
    print(line)
```

OUTPUT

```
python   135 min  2h 15m
stats     45 min  0h 45m
```

After: Same Output, Half the Reading

Three moves: rename, extract, delete the branch that was never different.

- totals, course, minutes: the names now match the design document.
- format_duration has one job and one home. The if/else is gone: 45 // 60 is already 0.
- Output identical, checks green, one commit: Refactor: extract format_duration.

AFTER

```
def format_duration(minutes):
    return f"{minutes // 60}h {minutes % 60:02d}m"

def format_totals(totals):
    ordered = sorted(totals.items(),
                     key=lambda item: item[1],
                     reverse=True)
    return [f"{course:<8}{minutes:>4} min  "
            f"{format_duration(minutes)}"
            for course, minutes in ordered]

totals = {"python": 135, "stats": 45}
for line in format_totals(totals):
    print(line)
```

OUTPUT

```
python   135 min  2h 15m
stats     45 min  0h 45m
```

Guard Clauses: Fail Early, Then Relax

ACT 3 · REFACTOR

Handle the bad cases at the top and leave. The happy path reads straight down.

- Nested ifs make you hold three conditions in your head at once.
- A guard clause: if something is wrong, raise or return now.
- The last line is the normal answer, unindented, where the eye expects it.

OUTPUT

```
45
```

BEFORE

```
def parse_minutes(text):
    if text.strip():
        if int(text) > 0:
            return int(text)
        else:
            raise ValueError("must be above 0")
    raise ValueError("must not be empty")
```

AFTER

```
def parse_minutes(text):
    if not text.strip():
        raise ValueError("must not be empty")
    minutes = int(text)
    if minutes <= 0:
        raise ValueError("must be above 0")
    return minutes

print(parse_minutes(" 45 "))
```

Duplication: Write It Once, Call It Twice

ACT 3 · REFACTOR

Two copies drift apart. One gets the bug fix. The other one goes to the demo.

- Two near-identical functions differ in one word. That word becomes a parameter.
- Fix a bug in one place and both callers get it.
- Rule of three: the second copy is a warning, the third is a job.

OUTPUT

```
python   135 min
python     2 h
```

BEFORE

```
def format_minutes(course, minutes):
    return f"{course:<8}{minutes:>4} min"

def format_hours(course, hours):
    return f"{course:<8}{hours:>4} h"

print(format_minutes("python", 135))
print(format_hours("python", 2))
```

AFTER

```
def format_amount(course, amount, unit):
    return f"{course:<8}{amount:>4} {unit}"

print(format_amount("python", 135, "min"))
print(format_amount("python", 2, "h"))
```

TRAP: Refactoring Blind

Refactoring without a check is rewriting with your eyes closed.

- No checks, no proof the output stayed the same. Run test_logic.py before and after.
- A mixed commit hides four decisions. Which line broke it? Every line is a suspect.
- After the slice runs: one move per commit, message starting with Refactor:.

RULE The word Refactor: in a message promises the checks passed before and after.

OUTPUT

```
one commit that hides four decisions
Implement F2 and rename stuff and fix bug

four commits that explain themselves
Implement load_records
Refactor: rename t to totals
Refactor: extract format_duration
Fix header row reaching int()
```

Practice 2: Tidy Without Breaking

Eight minutes. The file works. Make it readable without changing its output.

- Run labs/session2/tidy_lab.py: six PASS.
- Three moves: rename, constant, extract.
- Run after every move. Still six PASS? Next move. A FAIL? Undo the last move.
- Done early? A guard clause removes the nested if in goal_status.

TIP VS Code renames every use at once: right-click the name, then Rename Symbol (F2).

OUTPUT

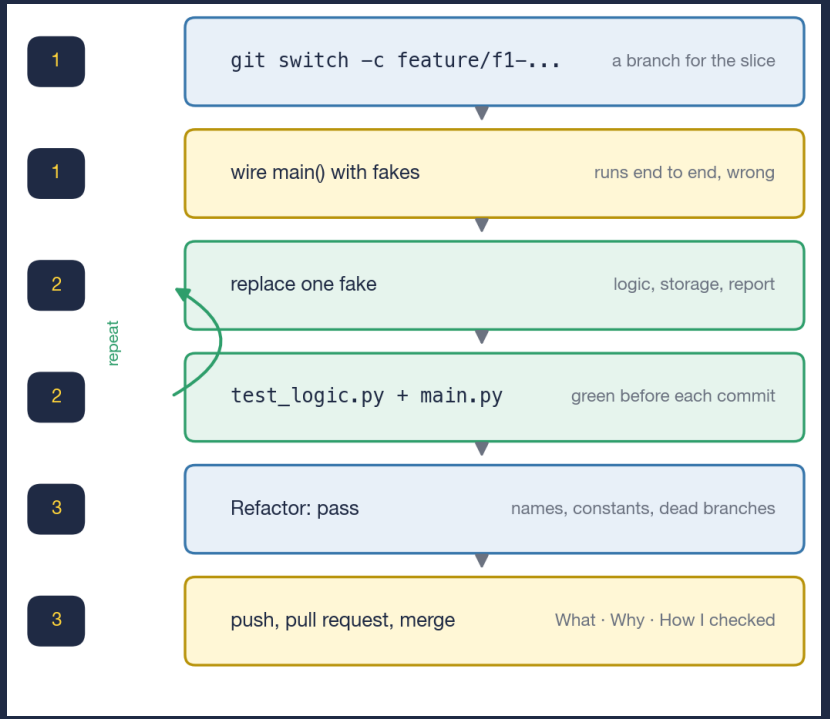
```
PASS daily_totals(sample)
PASS daily_totals([])
PASS goal_status(2100)
PASS goal_status(1500)
PASS format_report(sample)
PASS format_report([])
```

LAB

Lab: Ship Your F1

30 minutes. Your project, your branch, from stub to merged pull request.

- Step 1: branch and fake. Step 2: replace the fakes. Step 3: tidy, push, pull request.
- Sheet: labs/session2/slice-lab.md. Finished example: examples/study_logger_f1/.



When It Does Not Run

LAB

Read the last line of the traceback first. It names the problem. The line above names your file.

- No module named 'logic': wrong folder. cd into the project, then python3 main.py.
- NotImplementedError: make_record: a stub is still on F1's path. It is your next function.
- can only concatenate str (not "int"): a value is still text. int() it once, in logic.
- FileNotFoundError 'data/...': the data/ folder is missing. mkdir data, then run again.
- Nothing printed, no error: main.py lacks the if __name__ == "__main__": main() block.

TIP One suspect at a time. You replaced one fake, so the bug is in that function or its caller.

Lab Step 1: Branch and Fake

LAB

Six minutes. A branch for F1, and a main that runs end to end with fakes.

- git switch main, git pull, then git switch -c feature/f1-<short-name>.
- In main.py: ask for F1's inputs, call F1's functions by their design names, print the result.
- Give every function you have not written a fake body: return a constant of the right shape.
- Run python3 main.py. No traceback allowed. Then commit.

SHELL

```
git switch main
git pull
git switch -c feature/f1-save-entry
python3 main.py
git add -A
git commit -m "Wire main for F1 with fakes"
```

Lab Step 2: Replace the Fakes

LAB

Fourteen minutes. One function at a time, core first: logic, then storage, then report.

- Read the contract. Write the body. For a logic function, add one check to test_logic.py.
- Run test_logic.py, run main.py, commit "Implement <name>". Then the next fake.
- Stuck for more than three minutes? Spike it in scratch/, then come back.
- Do not touch F2 or F3. If a function is shared, implement only what F1 needs.

OUTPUT

```
one loop per function
1. body under the docstring
2. python3 test_logic.py   -> All checks passed.
3. python3 main.py         -> does the F1 job
4. git add -A && git commit -m "Implement <name>"
repeat until no fake is left on F1's path
```

Lab Step 3: Tidy, Push, Pull Request

LAB

Eight minutes. Refactor hat on, then ship.

- One refactor pass: design names, one constant for the file path, no dead branches. Commit as Refactor: ....
- git push -u origin feature/f1-<name>, then open the pull request with the four headings.
- Self-review the Files changed tab. Merge, delete the branch, git switch main, git pull.
- Submit: the merged pull request link and the output of git log --oneline.

OUTPUT

```
## What
F1 saves one study entry to data/study.csv
## Why
First Must feature, first slice of the MVP
## How I checked
python3 test_logic.py: All checks passed.
python3 main.py with 2026-11-10 / python / 30
and with -30: Not saved, file unchanged
## Not in this PR
F2 load_records, F3 weekly_totals (next session)
```

Five Ways to Lose Week 11

Each of these has turned a running MVP into a week 14 apology.

- Building layers: all of storage.py first, nothing to run until Thursday.
- Prototyping across the boundary: input() in logic, print() instead of raise.
- Refactoring without checks, or refactoring and adding a feature in one commit.
- Gold-plating: a menu, colors, a chart, before F1 saves a single row.
- One commit on Friday called "mvp". The grader cannot see how you got there.

Exit Check

Answer in one line each, then compare with a neighbour.

- What is a slice, and why does a slice beat a layer for a solo project?
- What does a fake return, and what stays real while the body is fake?
- Which two commands run before every commit in the lab?
- What makes a change count as refactoring, and what proves it?
- Name three of the five moves and the smell each one cures.

Submit and Next Session

F1 runs. Next session the other two Must features join it, and the MVP is complete.

- Submit via the LMS today: the merged pull request link and git log --oneline.
- Branch not merged yet? Push it and submit the open pull request link. Finish before next session.
- Next session: F2 and F3 as slices, then a refactor pass across all four files.
- End of next session: MVP demo. python3 main.py does every Must feature, start to finish.

> **RULE** One branch and one pull request per Must feature. Every commit is yours.